Views

In our examples up to this point, we have operated at the logical-model level.That is, we have assumed that the relations in the collection we are given are the actual relations stored in the database.

It is not desirable for all users to see the entire logical model. Security con-siderations may require that certain data be hidden from users. Consider a clerk who needs to know an instructor'sID, name and department name, but does not have authorization to see the instructor's salary amount. This person should see a relation described inSQL,by:

```
select ID, name, dept_name
from instructor;
```

Aside from security concerns, we may wish to create a personalized collection of relations that is better matched to a certain user's intuition than is the logical model. We may want to have a list of all course sections offered by the Physics department in the Fall 2009 semester, with the building and room number of each section. The relation that we would create for obtaining such a list is:

```
select course.course_id, sec_id, building, room_number
from course, section
where course.course_id = section.course_id and course.dept_name = 'Physics' and
section.semester = 'Fall' and section.year = '2009';
```

It is possible to compute and store the results of the above queries and then make the stored relations available to users. However, if we did so, and the underlying data in the relations instructor, course, or section changes, the stored query results would then no longer match the result of reexecuting the query on the relations. In general, it is a bad idea to compute and store query results such as those in the above examples (although there are some exceptions, which we will study later).

Instead, SQLallows a "virtual relation" to be defined by a query, and the relation conceptually contains the result of the query. The virtual relation is not precomputed and stored, but instead is computed by executing the query when-ever the virtual relation is used.

Any such relation that is not part of the logical model, but is made visible to a user as a virtual relation, is called a **view**. It is possible to support a large number of views on top of any given set of actual relations.

View Definition

We define a view in SQL by using the create view command. To define a view, we must give the view a name and must state the query that computes the view. The form of the create view command is:

create view v **as** <query expression>;

Where <query expression> is any legal query expression. The view name is represented by v.

Consider again the clerk who needs to access all data in the instructor relation, except salary. The clerk should not be authorized to access the instructor relation. Instead, a view relation faculty can be made available to the clerk, with the view defined as follows:

```
create view faculty as
select ID,name,dept_name
from instructor;
```

As explained earlier, the view relation conceptually contains the tuples in the query result, but is not precomputed and stored. Instead, the database system stores the query expression associated with the view relation. Whenever the view relation is accessed, its tuples are created by computing the query result. Thus,the view relation is created whenever needed, on demand.To create a view that lists all course sections offered by the Physics department in the Fall 2009 semester with the building and room number of each section, we write:

```
create view physics_fall_2009 as
select course.course_id,sec_id,building,room_number
from course,section
where course.course_id=section.course_id and course.dept_name='Physics' and section.semester='Fall' and section.year='2009';
```

Using Views in SQL Queries

Once we have defined a view, we can use the view name to refer to the virtual relation that the view generates. Using the view physics_fall_2009, we can findall Physics courses offered in the Fall 2009 semester in the Watson building by writing:

```
select course_id
from physics_fall_2009
where building='Watson';
```

View names may appear in a query any place where a relation name may appear. The attribute names of a view can be specified explicitly as follows:

```
create view departments_total_salary(dept_name,total_salary) as
select dept_name,sum(salary)
from instructor
group by dept_name;
```

The preceding view gives for each department the sum of the salaries of all the instructors at that department. Since the expression sum(salary) does not have a name, the attribute name is specified explicitly in the view definition.Intuitively, at any given time, the set of tuples in the view relation is the result of evaluation of the query expression that defines the view. Thus, if a view relation is computed and stored, it may become out of date if the relations used to define it are modified. To avoid this, views are usually implemented as follows.When we define a view, the database system stores the definition of the view itself, rather than the result of evaluation of the query expression that defines the view. Wherever a view relation appears in a query, it is replaced by the stored query expression. Thus, whenever we evaluate the query, the view relation is recomputed.
One view may be used in the expression defining another view. For example,we can define a view_physics_fall_2009_watson that lists the course_ID and room_number of all Physics courses offered in the Fall 2009 semester in the Watson building as follows:

```
create view_physics_fall_2009_watson as
select course_id, room_number
from physics_fall_2009
where building='Watson';
```